

Évaluation de Rattrapage PHP Objet : Gestion d'un Parc Automobile

Informations Clés

- **Durée** : 2 heures
 - **Modalité** : Travail individuel
 - **Technologie** : PHP 8+ avec typage strict (`declare(strict_types=1);`)
-

Contexte Métier

Une agence de location de véhicules souhaite structurer son application de back-office pour piloter sa flotte de transport. Vous êtes chargé de développer l'architecture principale en Programmation Orientée Objet (POO) en respectant un contrat d'interface rigoureux et des règles de calcul spécifiques à chaque type de véhicule.

3. Travail Demandé & Barème (20 points)

Partie 1 : La Classe Abstraite `Vehicule` (5 points)

Créez une classe abstraite `Vehicule` qui implémente `VehiculeInterface`.

1. **Encapsulation** : Déclarez les attributs protégés suivants : `immatriculation` (string), `marque` (string), `prixDeBase` (float), et `estDisponible` (bool).
2. **Constructeur** : Permet d'initialiser l'immatriculation, la marque et le prix de base. Par défaut, tout nouveau véhicule est considéré comme disponible (`true`).
3. **Méthodes** : Implémentez les getters pour chaque attribut ainsi que les méthodes issues de l'interface.
 - *Note* : La méthode `calculerTarifJournalier()` peut rester abstraite dans cette classe car la logique dépend entièrement du type final de véhicule.

Partie 2 : Les Classes Filles Spécifiques (6 points)

Développez deux classes héritant de la classe `Vehicule` :

1. `Voiture`

- Attribut spécifique privé : `nombreDePortes` (int).
- Constructeur : Doit faire appel au constructeur parent et initialiser le nombre de portes.
- Règle métier (`calculerTarifJournalier`) : Le tarif correspond au `prixDeBase`. Cependant, si la voiture possède **moins de 4 portes**, une réduction de **10%** est appliquée sur le tarif final.

2. `Camion`

- Attribut spécifique privé : `chargeUtileMax` (float, en tonnes).

- Constructeur : Doit faire appel au constructeur parent et initialiser la charge utile.
- Règle métier (`calculerTarifJournalier`) : Le tarif correspond au `prixDeBase` auquel on ajoute une majoration fixe de **20 € par tonne** de charge utile maximale.

Partie 3 : Le Gestionnaire de Flotte `GestionnaireParc` (5 points)

Créez une classe `GestionnaireParc` chargée de centraliser les opérations sur la flotte de véhicules.

1. **Attribut** : Un tableau privé `vehicules` (initialisé à vide).

2. **Méthodes** :

- `ajouterVehicule(VehiculeInterface $vehicule): void` : Ajoute un véhicule respectant l'interface au tableau.
- `getVehiculesDisponibles(): array` : Filtre et retourne uniquement la liste des objets véhicules actuellement disponibles.
- `calculerRevenuPotentielTotal(): float` : Parcourt l'ensemble du parc et calcule la somme des tarifs journaliers de *tous* les véhicules (qu'ils soient loués ou non), démontrant le concept de polymorphisme.

Partie 4 : Script de Test et Restitution `index.php` (4 points)

Dans un script `index.php`, mettez en œuvre votre architecture :

1. Instanciez un objet `GestionnaireParc`.
2. Ajoutez au moins **deux voitures** (une 3 portes et une 5 portes) et **un camion** avec des valeurs cohérentes.
3. Modifiez la disponibilité d'une des voitures pour simuler une location en cours (`false`).
4. Affichez proprement (via des échos textuels ou une structure HTML simple) :
 - La liste et le détail des véhicules actuellement disponibles.
 - Le chiffre d'affaires potentiel total généré par la flotte sur une journée de location.

Contraintes Techniques Strictes

- `declare(strict_types=1);` doit obligatoirement figurer sur la première ligne de chaque fichier PHP.
- Aucun attribut de classe ne doit être public (respect strict de l'encapsulation).
- Le code doit être correctement donné, typé et indenté.